CHAPTER 1

DESIGN A CHATGPT SYSTEM

1 System Design: ChatGPT System . . 2
2 Design of a ChatGPT System 6

Explore the design and architecture of a scalable ChatGPT-like conversational system.

1 System Design: ChatGPT System

1.1 What is ChatGPT?

ChatGPT is an advanced conversational application built on a sophisticated artificial intelligence language model developed by OpenAI. It is designed to understand natural language and generate human-like text, making it a highly effective tool for various communication tasks. Clear and effective communication is essential in various situations, including explaining complex concepts, generating ideas, and crafting professional content. In these cases, ChatGPT can help by:

- Simplifying technical subjects.
- Proposing innovative ideas.
- Composing clear, well-structured text.

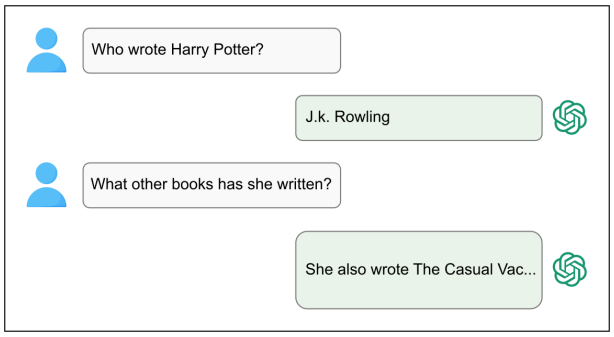


Figure 1.1: ChatGPT understands context and provides relevant follow-up responses

ChatGPT-like systems are particularly noteworthy because they can interpret various inputs, even when prompts are incomplete or ambiguous. They can respond to complex, abstract, or open-ended questions with clarity and relevance. These capabilities are made possible by sophisticated generative AI techniques, extensive training on large datasets, and advanced language modeling. Rather than relying on predefined answers, such systems analyze context, identify patterns in language, and generate responses that align with the intent and structure of the input.
In this chapter, we'll design a text-to-text generation system similar to ChatGPT. While OpenAI has not publicly disclosed the exact architecture and design details of ChatGPT, the approach covered in this lesson is based on general principles, established techniques, and best practices commonly used in building such systems.

Let's start by defining the core requirements, which play a critical role in shaping the overall design:

Requirements

Building the backend of a text-to-text generation system like ChatGPT requires careful attention to both functional and non-functional requirements:

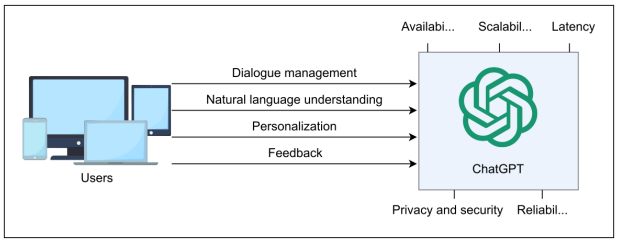
Functional requirements

- **Dialogue management:** The system must enable context-aware, text-based conversations by interpreting user input, maintaining context, and generating relevant, human-like responses.
- **Natural language understanding:** The system must interpret user input by identifying intent, extracting key entities, and understanding the context.

- **Personalization:** The system should adapt to user preferences and provide personalized recommendations based on previous interactions.
- **Feedback:** The system should improve its accuracy and performance over time based on feedback.

Non-functional requirements

- **Scalability:** The system must be able to handle increasing user traffic and workload.
- **Availability:** The system should be consistently available to users with minimal downtime.
- **Reliability:** The system should ensure consistent behavior, generating stable and predictable responses under varying conditions.
- **Low latency:** The system should generate responses quickly through efficient inference and caching without degrading the user experience.
- **Privacy and security:** The system should protect user data from unauthorized access and guarantee that communication and data storage comply with privacy standards and secure practices.



How does the system differentiate between interpreting incomplete or ambiguous user inputs? Use the AI assessment widget below to submit your solution and get an interactive response.

Question:

Incomplete user inputs

Reference Answer:

A conversational AI distinguishes incomplete or ambiguous inputs by analyzing the surrounding context and the user's previous messages. It applies intent recognition and confidence scoring to identify the most likely meaning behind unclear phrases. When confidence is low, the system may ask clarifying questions to ensure it interprets the user's intent accurately.

The next step in designing a system is to estimate the resources needed for its smooth operation. Let's discuss them in the following section:

1.2 Resource estimation

Let's break down the resource requirements for the ChatGPT system—the servers, storage, and bandwidth needed to handle a large user base. To keep our estimation practical and manageable, we'll use the following assumptions.

- ChatGPT has 1 billion users, with 150 million daily active users (DAU) interacting with the system.
- On average, each active user sends 10 requests daily, making it 1.5 billion requests daily.
- The size of each user request is approximately (The 1 KB request size is estimated based on typical request components: metadata (200–300 bytes) for user identification and timestamps, headers (300–400 bytes) for protocol-specific routing, user input (~400–500 bytes) assuming an average of 250–300 characters, and additional overhead (~100 bytes) for security and integrity checks.)2 KBRequestSize2kb(The 1 KB request size is estimated based on typical request components: metadata (200–300 bytes) for user identification and timestamps, headers (300–400 bytes) for protocol-specific routing, user input (~400–500 bytes) assuming an average of 250–300 characters, and additional overhead (~100 bytes) for security and integrity checks.), as most inputs are short text prompts.

- The average response generated by ChatGPT is 5 KB, considering that AI-generated replies are typically longer than user requests.

The number of servers estimation

We assume all users are active and making requests, resulting in 150 million requests per second. We 'll use the following formula to calculate the number of servers:

$$\text{Servers needed at peak load} = \frac{\text{Number of requests/second}}{\text{RPS of server}}$$

Using 64,000 as the estimated RPS a server can handle from the Back-of-the-envelope calculation section, we'll calculate the required number of servers as follows:

$$\text{Servers needed at peak load} = \frac{150 \text{ million}}{64,000} \approx \ 2343 \text{ servers}$$

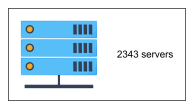


Figure 1.2: Number of servers required for ChatGPT

> **Note:** We've estimated the application servers needed for the ChatGPT system, but there's more to consider. Since generative AI relies on GPU servers for fast inferences, it's crucial to factor them in. For a quick estimation of GPU requirements, check out the back-of-the-envelope calculations.

Storage estimation

The required storage depends on the total number of requests and their size. We'll use the following formula to compute the storage:

$$\text{Total}_{\text{storage}} = \text{Total}_{\text{daily requests}} \text{Request size}$$

Below is a calculator to help us estimate our required resources.

> **Note:** You can change the value highlighted in yellow color to see the change in storage requirement.

- Total daily request = Total DAU × average daily request per user = 150 million × 10 = 1500 million
- Total storage = Total daily requests × request size = 1500 × 2 KB = 3 TB

Modern deep learning models are often massive, requiring significant storage and computational power. For example, a model with 3 billion parameters in FP16 (16-bit floating point) format occupies around 6 GB(= 3 billion x 2 bytes = 6 GB) of storage. To enable real-time inference, such models are typically loaded into GPU memory, and when dealing with even larger architectures, they are distributed across multiple GPUs to balance the computational load efficiently.

Bandwidth estimation

Bandwidth estimation is essential for understanding network resource requirements and ensuring efficient data transmission. It is calculated separately for incoming and outgoing traffic, as each depends on different factors.

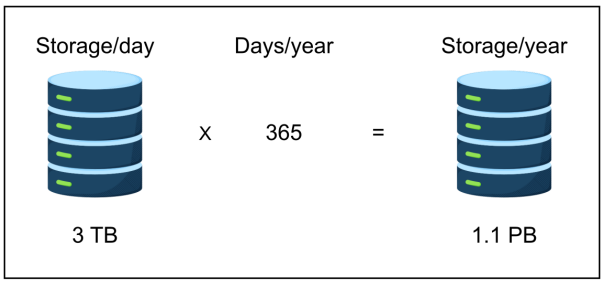


Figure 1.3: Total storage required by ChatGPT in a year

Incoming bandwidth Incoming bandwidth is determined by the number of incoming requests per second and the size of each request. Since most user requests to ChatGPT are relatively short, we assume an average request size of 2 KB, as mentioned above.
The total incoming bandwidth can be calculated using the formula:

$$\text{Total}_{\text{incoming bandwidth}} = \text{Total}_{\text{requests/sec}} \times \text{Total}_{\text{request size}}$$

Note: We can adjust the values in the table to see how the incoming bandwidth varies.

Outgoing bandwidth Outgoing bandwidth is determined by the number of responses per second and the size of each response. Since ChatGPT dynamically generates responses based on user input, the response size may vary. However, for estimation purposes, we assume an average response size of 5 KB per request.
The total outgoing bandwidth can be calculated using the formula:

$$\text{Total}_{\text{outgoing bandwidth}} = \text{Total}_{\text{responses/sec}} \times \text{Total}_{\text{response size}}$$

As user requests increase, the outgoing bandwidth consumption scales accordingly. Unlike incoming bandwidth, which is primarily influenced by short user requests, outgoing bandwidth is significantly higher due to the larger size of generated responses. We can calculate the outgoing bandwidth using the calculator given below:

Component type 'InstaCalc' not yet supported.

Note: We can adjust the values in the table to see how the values change.

1.3 Building blocks we will use

Now that we have completed the resource estimations, let's identify the essential building blocks to design a ChatGPT system. Below are the core components:

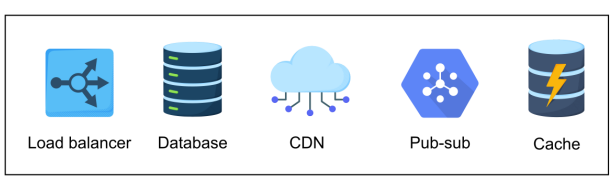


Figure 1.4: Essential building blocks required to design a ChatGPT system

1. **Load balancers** distribute user requests across multiple servers and services.
2. **Databases** are needed to store data in a graph format and associated metadata.

3. **CDNs** deliver content to end users, reducing delay and the burden on end-servers.
4. **A pub-sub system** manages real-time messaging and events within the system.
5. A **cache** is an important building block for keeping frequently accessed data.

In addition to the building blocks mentioned, other key components, such as an API gateway and ZooKeeper, play a crucial role in the system's design. We 'll discuss these in the next lesson as we dive into ChatGPT's System Design.

2 Design of a ChatGPT System

So far, we've identified the requirements, estimated storage needs, and outlined the foundational components for designing a ChatGPT-like system. Now , we'll move into its System Design to understand how these components work together and how it ensures real-time, context-aware conversations.

2.1 High-level design of ChatGPT

A system as complex as ChatGPT requires a well-structured design to handle real-time conversations efficiently. The high-level design provides a comprehensive overview of how components work together to form a cohesive system.

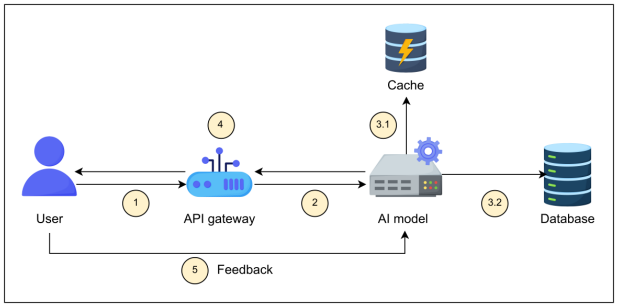


Figure 1.5: The high-level design of the ChatGPT system

The workflow for the high-level design is provided below:

1. The user submits a text prompt through the interface or an API.
2. The API gateway receives the request, handles authentication, rate limiting, and session management, and then forwards the prompt to the model server for processing.
3. The AI model processes the provided prompt and uses the conversation history to generate a response. These responses are stored in the cache to quickly retrieve repeated or similar requests and saved in the database for logging, analytics, or future reference.
4. The final response is sent back to the user via the API gateway.
5. User feedback is collected and stored to help improve system performance and guide future model updates or retraining.

Did you know that every piece of feedback you provide helps ChatGPT grow smarter? It is used to fine-tune the system by enhancing its ability to generate more accurate and natural responses.
Curious about how this process works? Dive deeper into "System Design of a Text-to-Text Generation System."

Quiz: Question 1: Is a typical cache (like LRU or TTL-based) sufficient for storing

AI-generated responses? **Answer:** While traditional caching strategies such as LRU or TTL are effective for static or frequently accessed web content, they are often inadequate for AI-generated responses, which are highly contextual and dependent on nuanced input factors such as prompt wording, user history, and personalization. To address this, ChatGPT-like systems benefit from a semantic cache that stores not only the response but also a vector embedding of the input prompt that captures its meaning. When a new prompt is received, the system can retrieve previous responses whose embeddings are semantically similar using a vector-based cache powered by approximate nearest neighbor (ANN) search. This enables the reuse of responses based on meaning rather than exact text matches. So hybrid approach proves to be more effective in maintaining performance,

relevance, and scalability.

With the high-level architecture in place, let's explore the detailed design:

2.2 Detailed design of ChatGPT

The high-level design gives us a broad overview, but the real magic lies in the detailed design, where we break down the technical intricacies of ChatGPT's architecture. Let 's explore each key component in depth before diving into how they come together in the system's workflow:

Components in the detailed design of ChatGPT

Each component plays a crucial role in making ChatGPT function seamlessly. The user's requests are routed through the API gateway to a load balancer that distributes traffic efficiently across application servers. The detailed design of the ChatGPT system involves the following components:

- **Pre-processing (NLU service):** This service processes user input by extracting meaning, intent, and key entities while converting text into numerical embeddings(An embedding is a vector that captures the meaning of text by placing semantically similar inputs close together in a numerical space.). These embeddings capture semantic relationships, enabling the model to generate contextually relevant responses.
- **Model server:** This is the core processing unit that hosts the AI models. It generates responses based on user queries.
- **Vector database:** It stores vector embeddings of past conversations and relevant external knowledge. It allows the system to retrieve semantically similar content, thereby facilitating context-aware responses.
- **Redis (caching layer):** It temporarily stores recent conversation history to avoid repeated database lookups and ultimately improves response time.

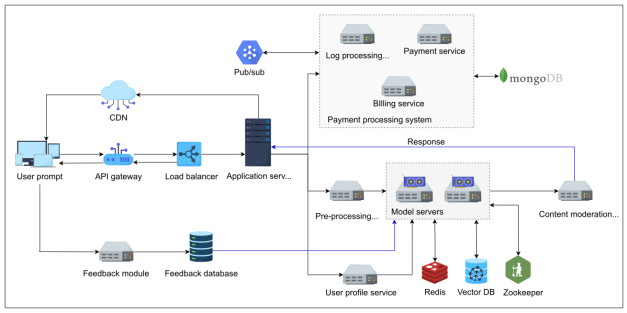


Figure 1.6: Detailed design of a text-to-text generation system like ChatGPT

- **Content moderation system:** This system scans responses for policy violations and inappropriate or sensitive content before delivering them to the user.
- **User profile service:** This service manages authentication while tracking user sessions and storing preferences. It ensures a personalized experience based on past interactions.
- **Zookeeper (orchestrator):** It coordinates model server management by tracking active instances, distributing workloads, and handling failovers. It ensures smooth service discovery and configuration updates for real-time request handling.
- **Pub/sub:** It manages asynchronous communication between various backend services, including the payment system, by ensuring that events like billing updates, logins, or request failures propagate without blocking API calls.
- **Payment processing system:** This system handles payment-related tasks such as logging transactions, generating bills, and processing payments, all coordinated through a pub/sub-system for seamless communication.
- **MongoDB (user database):** It stores structured data like session metadata, payment data, and billing history. This choice is driven by MongoDB's support for flexible data models, horizontal scalability, and high availability.

  Quiz: Question 1: How does the payment system determine how much to charge a user for a request in a system like ChatGPT, especially when billing is based on token usage or premium features? **Answer:** The payment system calculates charges based on usage logs/data, which track details like the number of tokens used, the complexity of the request, and any premium features accessed. After processing the request, the billing service computes the appropriate cost based on factors like token count or other pricing metrics. This information is then sent to the payment system for user approval to ensure that the user is fully informed and consents to the charge before finalizing the transaction.

Now, let's walk through the step-by-step journey of processing a user's prompt.

Workflow of the ChatGPT system

When a user enters a prompt through a web or mobile interface, the request first reaches the API gateway, which manages authentication, authorization, and rate limiting. From there, a load balancer distributes the request across multiple application servers to ensure efficient handling.
Before reaching the model, the prompt passes through the preprocessing NLU service. If the user has interacted before, a user profile service retrieves relevant historical data to personalize the response. Once preprocessed, the request is sent to the model servers, where the large language model (LLM) generates a response. When additional context is required, the system queries the vector database to retrieve semantically relevant information from past interactions.
Before the response is returned, it passes through the content moderation system to filter out harmful or inappropriate content. Zookeeper coordinates the model servers by managing configuration, leader election, and health monitoring during scaling and failover.
If the request involves a paid feature, the payment processing system logs the transaction and updates records in MongoDB. Meanwhile , a CDN ensures static assets, such as UI elements, load quickly, but generated responses from the model are handled separately.
Finally, the response is sent back to the user. If the user provides feedback, it is collected via the feedback module and stored in the feedback database. Structured feedback (like ratings or thumbs up/down) goes into a relational database (e.g., PostgreSQL or MySQL). In contrast, unstructured feedback (such as comments or session metadata) is stored in a document database like MongoDB. This data helps improve the system over time through analysis and model updates.
Why is content moderation placed after response generation rather than before?
Use the AI assessment widget below to submit your solution and get an interactive response.

Question:

Content moderation

Reference Answer:

Because moderation must evaluate the actual generated output to filter harmful or inappropriate content, which cannot be fully predicted before generation.

We've explored the detailed design and workflow of the ChatGPT system. Now it's time for the real test! Let's assess how effectively it meets the key requirements and performs under real-world conditions.

Achieving requirements

Let's see how our proposed ChatGPT system fulfills the functional and non-functional requirements that we identified in the previous lesson:

Achieving functional requirements A thoughtfully engineered system should primarily ensure that it meets its intended functionality. The table below illustrates how our design addresses the functional requirements to effectively fulfill the system's overall objectives.

Functional Requirements	Techniques
Dialogue management	• The pre-processing NLU service interprets user input. • The model server generates context-aware responses. • User profile service maintains the session context.
Natural language understanding	• The pre-processing NLU service extracts intent and entities and understands user input. • Vector DB enables semantic retrieval. • Redis caches recent context for fast access.
Personalization	• User profile stores preferences and history. • Model server uses this data for tailored responses.
Feedback	• Feedback module logs user ratings and comments, which are used to refine the system through offline training and model improvements.

Achieving non-functional requirements An effective system must not only operate correctly but also satisfy its non-functional requirements. These requirements define the system's performance, reliability, and scalability in real-world environments. The table below highlights how our design tackles the non-functional requirements:

2.3 Conclusion

In this chapter, we designed a ChatGPT-like system, covering everything from high-level architecture to detailed workflows. We saw how it ensures scalability, reliability, and security while delivering real-time, context-aware responses through components like caching, content moderation, and distributed model hosting.

But this is only the beginning.

- While we discussed the context-awareness module, how is that actually created and integrated into the system?
- How is user feedback incorporated in the subsequent responses generated by the GenAI System?

Non-functional Requirement	Techniques
Scalability	• The load balancers distribute traffic across multiple servers. • Model servers and vector database scale horizontally as demand grows. • Redis caching reduces redundant processing.
Reliability and availability	• Zookeeper coordinates distributed components for fault tolerance. • Redundant model servers and databases ensure continued operation in case of failures. • Pub-sub enables decoupled communication for real-time updates and seamless failure recovery.
Low latency	• Redis caching accelerates responses for frequently queried data. • Vector database efficiently retrieves contextual information, reducing unnecessary recomputation. • CDN reduces latency by caching and serving static content closer to users.
Security and privacy	• API gateway enforces authentication, authorization, and rate limiting. • Payment processing system ensures secure financial transactions. • End-to-end encryption secures requests and responses from interception. • MongoDB stores user data with encryption and access control.

To develop a deeper understanding of the questions above, explore Grokking the Generative AI System Design.
Looking ahead, advances in LLMs, RAG (Retrieval-augmented generation), and efficient inference will enhance performance, while trends such as multimodal AI and on-device inference may make ChatGPT-like systems more responsive and personalized.